

راهنمای جامع شبیه‌سازی پرواز پرندگان (Boids) و هوش جمعی خودسامانده روی موبایل

آزمایشگاه محاسباتی در جیب — اپیزود ۲

۱. مقدمه و پیش‌زمینه علمی

در بسیاری از پدیده‌های طبیعی، سیستم‌هایی وجود دارند که رفتارهای فوق‌العاده پیچیده، یکپارچه و به ظاهر هوشمندانه‌ای را به نمایش می‌گذارند، بدون آنکه هیچ مرکز فرماندهی، ژنرال یا رهبری برای هدایت آن‌ها وجود داشته باشد. این پدیده در نظریه سیستم‌های پیچیده «برآیش» و «خودساماندهی» (Emergence / Self-Organization) نامیده می‌شود.

هوش جمعی بدون رهبر چیست؟

حرکت هزاران پرنده سار در آسمان (Murmuration)، مانور صدها ماهی در برابر شکارچی بدون برخورد با یکدیگر، یا ساخت سازه‌های عظیم توسط موریانه‌ها، نمونه‌های کلاسیک این پدیده‌اند. در این ساختارها:

- هیچ پرنده‌ای کل نقشه پرواز دسته را در ذهن ندارد.
 - هیچ پیام رادیویی، رهبری مرکزی یا تله‌پاتی مرموزی در کار نیست.
 - هر فرد صرفاً به اطلاعات حسی بسیار محدود از همسایه‌های مجاور خود پاسخ می‌دهد.
- نظم سراسری و هوش جمعی، نه از یک دستور از بالا به پایین (Top-Down)، بلکه از برهم‌کنش موضعی و پایین به بالا (Bottom-Up) متولد می‌شود.

الگوریتم بویدز (Craig Reynolds, 1986)

در سال ۱۹۸۶، دانشمند علوم کامپیوتر «کریگ رینولدز» مدلی تحت عنوان بویدز (Boids - برگرفته از کلمه عامیانه Bird-oid به معنی پرنده‌نما) را توسعه داد. او نشان داد که برای بازسازی ریاضی و گرافیکی رفتار پیچیده پرندگان، نیازی به محاسبات پیچیده هوش مصنوعی نیست؛ بلکه اعمال ۳ قانون فوق‌العاده ساده بر روی سرعت و موقعیت هر ذره برای خلق یک گله هماهنگ کاملاً کافی است. این مدل بعدها در صنعت هالیوود و

پوریا آزادی

انیمیشن سازی (از جمله صحنه تاریخی رمیدن گله گاومیش ها در انیمیشن شیر شاه دیزنی) به استاندارد شبیه سازی گله ها تبدیل شد.

۲. مدل سازی ریاضی و دینامیک برداری

برای مدل سازی یک دسته شامل N پرنده نما (بوید)، وضعیت هر بوید i با دو بردار موقعیت $\mathbf{x}_i \in \mathbb{R}^2$ و بردار سرعت $\mathbf{v}_i \in \mathbb{R}^2$ در فضای دو بعدی توصیف می شود.

هندسه همسایگی

هر پرنده فقط محیط اطراف خود را تا یک فاصله مشخص می بیند. بنابراین مجموعه همسایگان پرنده i (N_i) به صورت زیر تعریف می شود:

$$N_i = \{j \mid 0 < \|\mathbf{x}_j - \mathbf{x}_i\| \leq R_{visual}\}$$

همچنین یک حوزه حریم امن یا حداقل فاصله ($R_{sep} < R_{visual}$) برای جلوگیری از برخورد فیزیکی تعریف می شود.

قوانین سه گانه رینولدز

$$\mathbf{F}_{total} = w_{sep}\mathbf{F}_{sep} + w_{ali}\mathbf{F}_{ali} + w_{coh}\mathbf{F}_{coh}$$

۱. جداسازی (Separation) — قانون دفع برخورد

پرنده باید از برخورد با همسایگانی که بیش از حد نزدیک شده اند اجتناب کند. نیروی دافعه با معکوس فاصله نسبت عکس دارد تا مانع تصادف شود:

$$\mathbf{F}_{sep}^{(i)} = - \sum_{j \in N_i^{close}} \frac{\mathbf{x}_j - \mathbf{x}_i}{\|\mathbf{x}_j - \mathbf{x}_i\| + \epsilon}$$

۲. هم راستایی (Alignment) — تطابق بردار سرعت

پرنده تلاش می کند جهت حرکت و سرعت خود را با میانگین بردار سرعت همسایگان مجاور هم فاز کند:

$$\mathbf{F}_{ali}^{(i)} = \left(\frac{1}{|N_i|} \sum_{j \in N_i} \mathbf{v}_j \right) - \mathbf{v}_i$$

پوریا آزادی

۳. انسجام (Cohesion) — کشش به مرکز ثقل محلی

برای جلوگیری از متلاشی شدن گله و جدا افتادن اعضا، پرنده به سمت مرکز جرم هندسی همسایگان جذب می‌شود:

$$\mathbf{F}_{coh}^{(i)} = \left(\frac{1}{|N_i|} \sum_{j \in N_i} \mathbf{x}_j \right) - \mathbf{x}_i$$

محدودسازی نیروها و مرزهای نرم (Soft Boundaries)

در فیزیک واقعی، عضلات پرنده توان شتاب‌گیری و اعمال نیروی نامحدود ندارند؛ بنابراین بردار شتاب نهایی توسط عملگر اشباع (Clamping) محدود می‌شود:

$$\|\mathbf{F}\| \leq F_{max}, \quad \|\mathbf{v}\| \leq v_{max}$$

همچنین برای اینکه پرندگان درون کادر نمایش عمودی ۹:۱۶ گوسی باقی بمانند و پروازی طبیعی داشته باشند، یک نیروی نرم دور شونده از دیواره‌ها (Repulsive Border Force) در حاشیه کادر تعبیه شده است.

سنجه ریاضی هماهنگی جمعی (Polarization Order Parameter)

برای اینکه بتوانیم روند تغییر فاز از بی‌نظمی به نظم سراسری را اندازه‌گیری کنیم، از پارامتر قطبش (Φ) استفاده می‌کنیم:

$$\Phi(t) = \frac{\left\| \sum_{i=1}^N \mathbf{v}_i(t) \right\|}{\sum_{i=1}^N \left\| \mathbf{v}_i(t) \right\|} \times 100\%$$

- اگر بردارها در جهات رندوم و مخالف هم باشند، $\Phi \approx 0\%$ است (آشوب و بی‌نظمی).
- اگر تمام پرندگان در یک راستا و با هم مانور دهند، $\Phi \approx 100\%$ خواهد بود (نظم سراسری).
-

۳. پیش‌نیازها و راه‌اندازی بستر روی گوسی

این شبیه‌سازی به صورت سبک و وکتورایز شده (Vectorized) با NumPy پیاده شده و به پردازش سنگین نیازی ندارد.

پوریا آزادی

مسیر اول: ترموکس در اندروید (آفلاین و بومی)

وارد توزیع لینوکس اوبونتو شوید:

```
Bash
```

```
proot-distro login ubuntu
```

در صورت نیاز، پکیج‌های پایتون و رسم نمودار را بررسی یا نصب کنید:

```
Bash
```

```
apt update && apt install python3 python3-numpy python3-matplotlib  
python3-pil -y
```

مسیر دوم: گوگل کولب (iOS و مرورگر)

کاربران آیفون یا کامپیوتر می‌توانند بدون نصب هیچ نرم‌افزاری، یک نوت‌بوک جدید در colab.research.google.com ایجاد کرده و کد مرحله بعد را در آن اجرا کنند.

۴. کد کامل پایتون (نسخه ۲۰ ثانیه‌ای عمودی با گیج HUD)

اسکرپت زیر شبیه‌سازی را برای دقیقاً ۲۰ ثانیه (۵۰۰ فریم با نرخ ۲۵ فریم بر ثانیه) در نسبت ابعاد ۹:۱۶ اجرا می‌کند. رنگ هر پرنده به‌طور زنده بر اساس میزان هماهنگی با اطرافش از نارنجی (بی‌نظم) به فیروزه‌ای نئونی (هماهنگ) تغییر کرده و وضعیت گله در HUD بالای صفحه ثبت می‌شود:

```
Python
```

```
import numpy as np  
import matplotlib.pyplot as plt  
from matplotlib.animation import FuncAnimation, PillowWriter  
  
# =====  
# 1. SIMULATION SETUP & CONSTANTS (20 SECONDS)  
# =====  
FPS = 25  
DURATION_SEC = 20  
TOTAL_FRAMES = DURATION_SEC * FPS # Exactly 500 frames
```

```
NUM_BOIDS = 130
WIDTH, HEIGHT = 100.0, 177.7    # Exact 9:16 vertical ratio

# Behavioral Radii & Kinematics
VISUAL_RADIUS = 16.0
MIN_DISTANCE = 4.2
MAX_SPEED = 2.6
MAX_FORCE = 0.14

W_SEP = 1.8 # Rule 1: Separation (Avoid collisions)
W_ALI = 1.1 # Rule 2: Alignment (Match neighbors' heading)
W_COH = 0.9 # Rule 3: Cohesion (Stay close to group center)

# Initialize positions and randomized velocities
np.random.seed(42)
positions = np.random.rand(NUM_BOIDS, 2) * [WIDTH, HEIGHT]
angles = np.random.rand(NUM_BOIDS) * 2 * np.pi
velocities = np.column_stack([np.cos(angles), np.sin(angles)]) * MAX_SPEED

# =====
# 2. NUMERICAL KINEMATICS ENGINE
# =====
def update_physics(pos, vel):
    diff = pos[np.newaxis, :, :] - pos[:, np.newaxis, :]
    dist = np.linalg.norm(diff, axis=2)

    in_range = (dist > 0) & (dist < VISUAL_RADIUS)
    too_close = (dist > 0) & (dist < MIN_DISTANCE)

    sep_steer = np.zeros_like(vel)
    ali_steer = np.zeros_like(vel)
    coh_steer = np.zeros_like(vel)
    local_order = np.zeros(NUM_BOIDS)

    for i in range(NUM_BOIDS):
        neighbors = in_range[i]
        close_neighbors = too_close[i]

        # 1. Separation
        if np.any(close_neighbors):
            repulsion = -diff[i, close_neighbors] / (dist[i, close_neighbors, np.newaxis] + 1e-4)
            sep_steer[i] = np.sum(repulsion, axis=0)

        # 2. Alignment & 3. Cohesion
        if np.any(neighbors):
            avg_vel = np.mean(vel[neighbors], axis=0)
            ali_steer[i] = avg_vel - vel[i]
            coh_steer[i] = np.mean(diff[i, neighbors], axis=0)

    # Local alignment index for each boid (0.0 = chaotic, 1.0 = aligned)
```

```
v_norm = vel[i] / np.linalg.norm(vel[i])
n_norms = vel[neighbors] / (np.linalg.norm(vel[neighbors], axis=1, keepdims=True) + 1e-5)
local_order[i] = max(0.0, np.mean(np.dot(n_norms, v_norm)))

# Force clamping helper
def limit(vectors, max_val):
    norms = np.linalg.norm(vectors, axis=1, keepdims=True)
    norms[norms == 0] = 1.0
    return np.where(norms > max_val, (vectors / norms) * max_val, vectors)

sep = limit(sep_steer, MAX_FORCE) * W_SEP
ali = limit(ali_steer, MAX_FORCE) * W_ALI
coh = limit(coh_steer, MAX_FORCE) * W_COH

# Soft Boundary Force (turn smoothly away from edges)
boundary_force = np.zeros_like(vel)
margin = 12.0
turn_factor = 0.28

boundary_force[pos[:, 0] < margin, 0] += turn_factor
boundary_force[pos[:, 0] > WIDTH - margin, 0] -= turn_factor
boundary_force[pos[:, 1] < margin, 1] += turn_factor
boundary_force[pos[:, 1] > HEIGHT - margin, 1] -= turn_factor

acceleration = sep + ali + coh + boundary_force
new_vel = limit(vel + acceleration, MAX_SPEED)
new_pos = pos + new_vel

# Hard clamp fallback
new_pos[:, 0] = np.clip(new_pos[:, 0], 0, 1, WIDTH - 1)
new_pos[:, 1] = np.clip(new_pos[:, 1], 0, 1, HEIGHT - 1)

return new_pos, new_vel, local_order

# Precomputing the entire 20-second trajectory
print(f"Calculating flocking physics ({DURATION_SEC}s, {TOTAL_FRAMES} frames)...")
pos_seq = np.zeros((TOTAL_FRAMES, NUM_BOIDS, 2))
vel_seq = np.zeros((TOTAL_FRAMES, NUM_BOIDS, 2))
order_seq = np.zeros((TOTAL_FRAMES, NUM_BOIDS))
global_order_metric = np.zeros(TOTAL_FRAMES)

curr_pos = positions.copy()
curr_vel = velocities.copy()

for f in range(TOTAL_FRAMES):
    pos_seq[f] = curr_pos
    vel_seq[f] = curr_vel
    curr_pos, curr_vel, local_ord = update_physics(curr_pos, curr_vel)
    order_seq[f] = local_ord
```

پوریا آزادی

```
# Global Polarization Order Parameter (0% to 100%)
mean_velocity = np.mean(curr_vel, axis=0)
avg_speed = np.mean(np.linalg.norm(curr_vel, axis=1))
global_order_metric[f] = (np.linalg.norm(mean_velocity) / (avg_speed + 1e-5)) * 100.0

# =====
# 3. HIGH-CONTRAST MOBILE RENDERER (9:16)
# =====
fig, ax = plt.subplots(figsize=(6, 10.66), facecolor="#020202")
ax.set_facecolor("#020202")
ax.set_xlim(0, WIDTH)
ax.set_ylim(0, HEIGHT)
ax.axis("off")

# Colormap: Orange/Magenta (Chaos) to Cyan/Aqua (Synchronized Flock)
cmap = plt.cm.plasma

# Directional Arrow Boids
quiver = ax.quiver(
    pos_seq[0, :, 0], pos_seq[0, :, 1],
    vel_seq[0, :, 0], vel_seq[0, :, 1],
    order_seq[0],
    cmap=cmap, clim=[0.0, 1.0],
    scale=34, width=0.008,
    headwidth=4.5, headlength=5.0, headaxislength=4.5,
    pivot='mid', alpha=0.95
)

# Glow Halo Points
glow = ax.scatter(
    pos_seq[0, :, 0], pos_seq[0, :, 1],
    c=order_seq[0], cmap=cmap, vmin=0.0, vmax=1.0,
    s=55, alpha=0.35, edgecolors='none'
)

# --- SCIENTIFIC HUD OVERLAY ---
hud_title = ax.text(0.05, 0.95, "EPISODE 02 // BOIDS EMERGENCE", transform=ax.transAxes,
                    color="#FFFFFF", fontsize=11, fontfamily="monospace", weight="bold")

hud_timer = ax.text(0.05, 0.92, "", transform=ax.transAxes,
                    color="#A0A0A0", fontsize=9, fontfamily="monospace")

hud_order = ax.text(0.05, 0.89, "", transform=ax.transAxes,
                    color="#00E5FF", fontsize=10, fontfamily="monospace", weight="bold")

hud_state = ax.text(0.05, 0.86, "", transform=ax.transAxes,
                    color="#FFD700", fontsize=9, fontfamily="monospace")

# Bottom rules indicator
ax.text(0.05, 0.03, "RULES: 1.Separate 2.Align 3.Cohere", transform=ax.transAxes,
```

```
color="#666666", fontsize=8, fontfamily="monospace")

def update(frame):
    pos = pos_seq[frame]
    vel = vel_seq[frame]
    ord_val = order_seq[frame]
    current_time = frame / FPS
    order_pct = global_order_metric[frame]

    # Update particles and directional arrows
    quiver.set_offsets(pos)
    quiver.set_UVC(vel[:, 0], vel[:, 1])
    quiver.set_array(ord_val)

    glow.set_offsets(pos)
    glow.set_array(ord_val)

    # Dynamic HUD text updates
    hud_timer.set_text(f"TIME: {current_time:04.1f}s / {DURATION_SEC}.0s")

    # Visual bar for Order percentage
    bar_length = int(order_pct / 10)
    bar_display = "[" + "#" * bar_length + "-" * (10 - bar_length) + "]"
    hud_order.set_text(f"SWARM ORDER: {order_pct:04.1f}% {bar_display}")

    # Phase diagnosis
    if current_time < 3.5:
        hud_state.set_text("PHASE: 1. RANDOM DISPERSION (CHAOS)")
        hud_state.set_color("#FF5555")
    elif current_time < 9.0:
        hud_state.set_text("PHASE: 2. LOCAL SYNC (CLUSTER FORMATION)")
        hud_state.set_color("#FFAA00")
    else:
        hud_state.set_text("PHASE: 3. GLOBAL SWARM INTELLIGENCE")
        hud_state.set_color("#00FFCC")

    return quiver, glow, hud_timer, hud_order, hud_state

ani = FuncAnimation(fig, update, frames=TOTAL_FRAMES, interval=1000/FPS, blit=True)

output_name = "boids_flocking_20s.gif"
print(f"Rendering 500 frames to {output_name}...")
ani.save(output_name, writer=PillowWriter(fps=FPS))
print(f"Export successful: {output_name}")
```


پوریا آزادی

۵. راهنمای گام به گام اجرا در ترموکس و انتقال فایل

۱. ساخت فایل در لینوکس اوبونتو:

```
Bash  
nano boids.py
```

۲. ذخیره سازی:

کد بالا را کپی و پیست کرده، کلیدهای Ctrl + O سپس Enter و نهایتاً Ctrl + X را فشار دهید.

۳. اجرا و رندر انیمیشن:

```
Bash  
python3 boids.py
```

ابتدا محاسبات ماتریسی فیزیک در کسری از ثانیه تکمیل شده و سپس رندر ۵۰۰ فریم گیف آغاز می شود تا پیام Export successful چاپ شود.

۴. انتقال به پوشه دائلود گوشی (مشاهده در گالری):

```
Bash  
exit  
cp ~/ubuntu/root/boids_flocking_20s.gif /sdcard/Download/
```

۶. تحلیل فیزیکی خروجی شبیه سازی

پس از باز کردن انیمیشن، این سه مرحله گذار فاز را زیر نظر بگیرید:

Plaintext		
[0.0s - 3.5s]	[3.5s - 9.0s]	[9.0s - 20.0s]
(Chaos) بی نظمی تصادفی	--> (Clusters) خوشه بندی محلی	--> (Flock) هوش جمعی یکپارچه
Order: 10% - 25% (رنگ های گرم)	Order: 40% - 75% (تغییر رنگ)	Order: 90%+ (فیروزه های نئونی)

پوریا آزادی

۱. فاز ۱: پراکندگی تصادفی (ثانیه ۰ تا ۳.۵):

بویدها با زاویه‌های تصادفی رها شده‌اند. برخوردها زیاد است و بردار کلی سرعت میانگین به صفر نزدیک است (شاخص نظم زیر ۲۵٪). پرندگان با رنگ بنفش و نارنجی تیره نمایش داده می‌شوند.

۲. فاز ۲: تشکیل خوشه‌های خودمختار محلی (ثانیه ۳.۵ تا ۹.۰):

نیروی انسجام و هم‌راستایی شروع به غلبه می‌کند. دسته‌های کوچک ۳ تا ۱۰ تایی تشکیل می‌شوند و رنگ آن‌ها به سمت زرد و فیروزه‌ای متمایل می‌شود. شاخص هماهنگی با شیب تند بالا می‌رود.

۳. فاز ۳: قفل فاز سراسری و هوش جمعی (ثانیه ۹.۰ تا ۲۰):

خوشه‌ها به هم پیوسته و کل ۱۳۰ پرنده به یک ارگانیسم واحد و موج تبدیل می‌شوند. شاخص نظم روی بالای ۹۰٪ تثبیت شده و کل گله با رنگ فیروزه‌ای هماهنگ در کادر دور می‌زند.

۷. آزمایش‌ها و چالش‌های محاسباتی

برای کاوش بیشتر در مکانیک سیستم‌های پیچیده، پارامترهای کد را تغییر دهید:

• چالش ۱: آزمایش فروپاشی گله (Breakdown Test)

وزن هم‌راستایی را صفر کنید ($W_ALI = 0.0$). بررسی کنید که آیا بدون هم‌راستایی، دو قانون دیگر می‌توانند ساختار گله را نگه دارند، یا پرندگان تبدیل به یک توده‌ی سرگردان و بی‌جهت می‌شوند؟

• چالش ۲: شبیه‌سازی گله‌های ترسو (Over-Separation)

فاصله امن MIN_DISTANCE را به دو برابر افزایش دهید. ثبت کنید که این تغییر چگونه مانع از به هم پیوستن خوشه‌های محلی می‌شود.

• چالش ۳: چگالی بحرانی (Critical Density)

پوریا آزادی

تعداد بویدها را از ۱۳۰ به ۳۰ کاهش دهید (NUM_BOIDS = 30). بررسی کنید که آیا در چگالی پایین، پرندگان اصلاً قادر به پیدا کردن یکدیگر و تشکیل هوش جمعی هستند یا خیر؟

این مستند به عنوان بخش دوم پروژه متن‌باز **Pocket Lab** منتشر شده است. سورس‌کدها، نوت‌بوک تعاملی برای اجرای ابری و اطلاعات تکمیلی در مخزن گیت‌هاب پروژه و کانال تلگرام در دسترس است:

<https://github.com/poria-azadi96/pocket-lab>

https://t.me/the_maze2022